

SPI SLAVE WITH SINGLE-PORT RAM

RTL DESIGN & VERIFICATION PROJECT REPORT

Project Title:	SPI SLAVE WITH SINGLE PORT RAM
Prepared By:	Ahmed Goda Sharawy
Supervisor:	Eng.Kareem Waseem
Department:	Communication & Electronics
Tools Used:	Questasim ,Questa Linting, Xilinx Vivado
Date:	9 / 8 / 2026

1. Project Introduction

This project implements an SPI Slave interface integrated with a Single-Port RAM, designed and verified using RTL (Register Transfer Level) methodology in Verilog. The system enables an external SPI Master device to write data into memory and read data back from memory through a standard SPI communication protocol.

An SPI Slave is a digital hardware block that receives commands and data serially from a master device and responds according to the SPI protocol timing and control signals (MOSI, MISO, SS_N, and CLK). In this project, the SPI Slave acts as the communication bridge between the external SPI Master and the internal memory.

The Single-Port RAM is used as the internal data storage element of the design. It allows a single read or write access at a time, and is controlled entirely by the SPI Slave module based on the decoded commands received over the SPI bus.

The SPI Slave communicates with the RAM by decoding incoming SPI commands, extracting the target address and/or data, and driving the RAM's address and data ports accordingly. When a read operation is requested, the RAM's output data is captured by the SPI Slave and shifted out serially through MISO back to the master.

The overall purpose of this project is to demonstrate a complete, functional memory-access system controlled over a standard serial protocol, suitable for use in embedded systems, sensor interfacing, or configuration/storage applications. The design was implemented at the RTL level using Verilog HDL and verified using a dedicated testbench and functional simulation to confirm correct write and read behavior before proceeding to synthesis and implementation.

Project Block Diagram

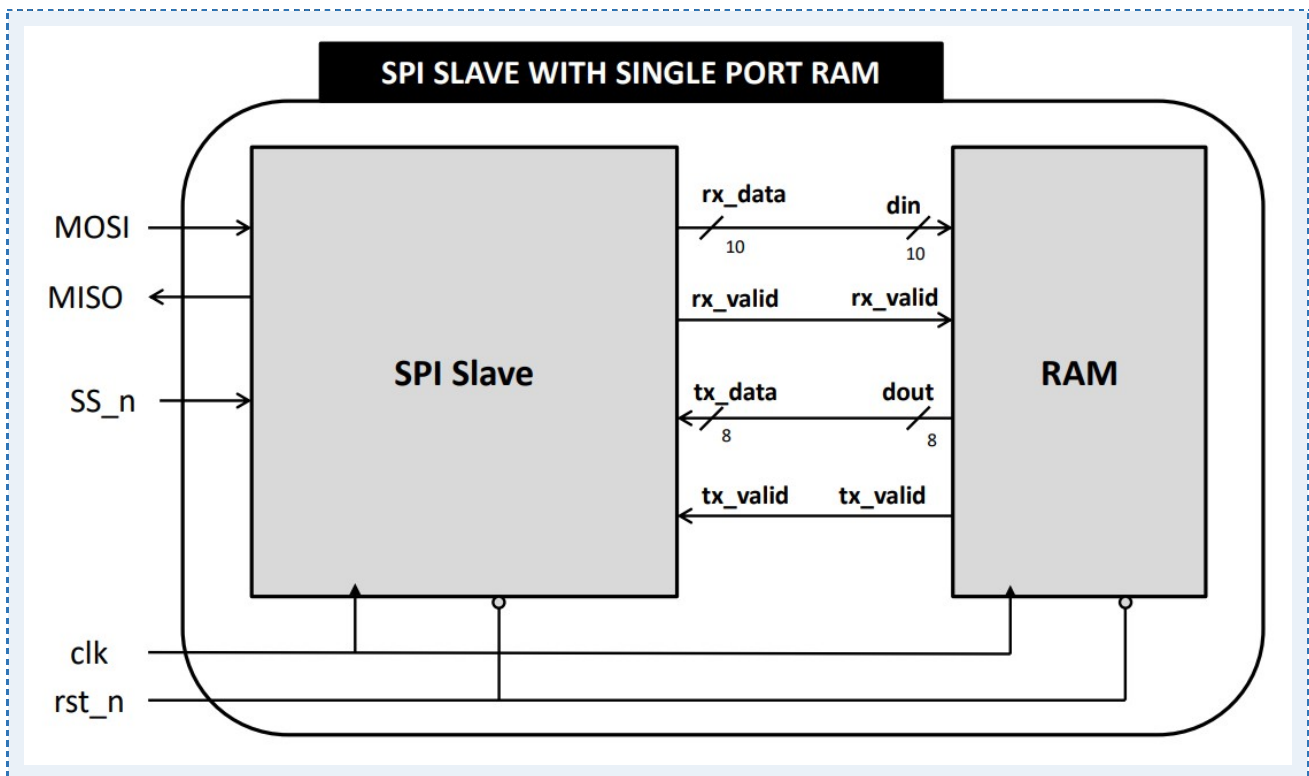


Figure 1.1 — High-level architecture showing SPI Master → SPI Slave → Single-Port RAM, including the SPI signals MOSI, MISO, SS_N, CLK, and RESET.

2. Project Architecture / Modules Overview

Before examining each block in detail, this section provides a general overview of the modules that make up the design and how they interact with one another.

Table 2.1 — Design Modules Overview

Module	Purpose
SPI Slave	Handles SPI communication and controls the protocol
Single-Port RAM	Stores and retrieves data
TOP	Connects the SPI Slave and RAM together
Testbench	Verifies the complete design

Overall Data Flow

The SPI Master initiates a transaction by pulling `SS_N` low and sending a serial command on `MOSI`, synchronized with `CLK`. The SPI Slave receives and decodes this command, then either drives an address/data value into the RAM (for write operations) or retrieves data from the RAM (for read operations) and shifts it out on `MISO`. The TOP module wires the SPI Slave and RAM together, and `RESET` is used to bring the system to a known idle state.

SPI Command Structure

The design uses a 2-bit command field to determine the type of operation being performed. The command is decoded by the SPI Slave at the start of each transaction:

Command Code	Operation
00	Write Address
01	Write Data
10	Read Address
11	Read Data

A typical write sequence begins with a Write Address command (00) to select the target RAM location, followed by a Write Data command (01) to store the value at that address. Similarly, a typical read sequence begins with a Read Address command (10) to select the target location, followed by a Read Data command (11) to retrieve the stored value through `MISO`.

3. SPI Slave Module

The SPI Slave module is the core communication block of the design. It is responsible for receiving serial commands and data from the SPI Master, decoding the requested operation, interfacing with the RAM, and returning read data back to the master.

3.1 FSM State Description

The SPI Slave operation is controlled by a finite state machine consisting of the following states:

State	Description
IDLE	Default state while SS_N is high (inactive). The slave waits for a new transaction to begin.
CHK_CMD	The first two received bits (command field) are checked to determine the requested operation (Write Address, Write Data, Read Address, or Read Data).
WRITE	The received parallel data is written into the RAM at the previously latched address, or the address itself is latched, depending on the decoded command.
READ_ADD	The received address is latched and used to access the RAM location whose data will subsequently be read.
READ_DATA	Data retrieved from the RAM is loaded into the shift register and transmitted serially to the master via MISO.

3.2 Command Decoding

- 00 = WRITE ADDRESS — latches the incoming address value for a subsequent write.
- 01 = WRITE DATA — writes the incoming data value into RAM at the latched address.
- 10 = READ ADDRESS — latches the incoming address value for a subsequent read.
- 11 = READ DATA — reads the RAM at the latched address and transmits the value back via MISO.

SPI Slave Block Diagram

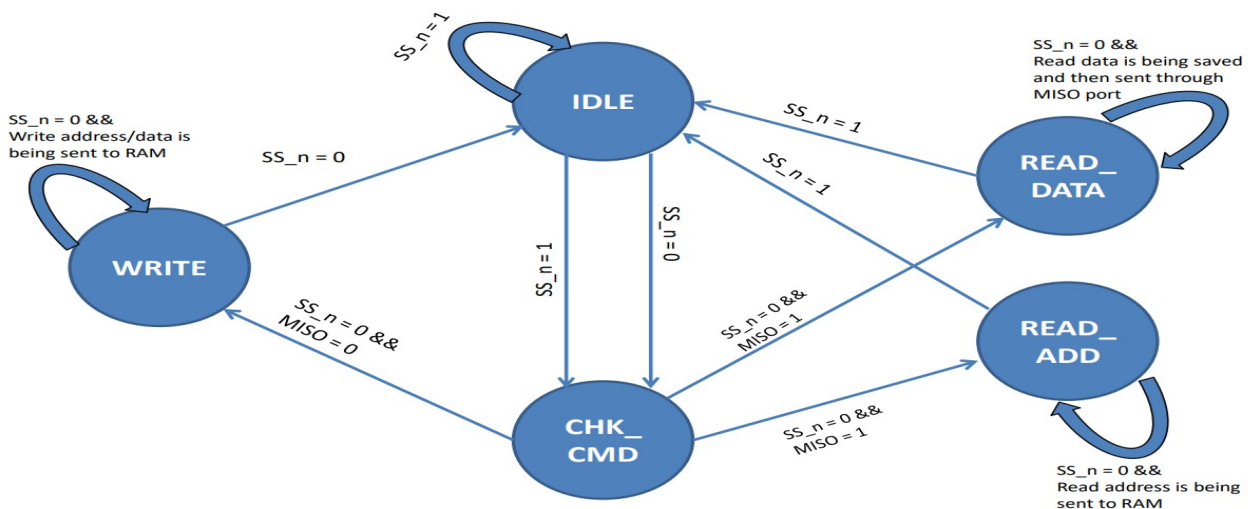


Figure 3.1 — Internal block diagram of the SPI Slave FSM state machine module.

3.3 SPI Slave RTL Code

```

2 > VERILOG > SPI.v > SPI
1  module SPI #(parameter IDLE = 3'b000,
2      parameter CHK_CMD = 3'b001,
3      parameter WRITE = 3'b010,
4      parameter READ_ADD = 3'b011,
5      parameter READ_DATA = 3'b100
6  )(
7  input  clk,rst_n,ss_n,
8  input  [7:0]tx_data,
9  input  tx_valid,
10 input  MOSI,
11 output MISO,
12 output reg [9:0]rx_data,
13 output reg rx_valid
14 );
15 // -----STATES-----
16
17 reg [2:0]ns,cs;
18
19
20 // internal signal FF for delay one cycle
21 reg cmd_first;
22 // block 3 SIPO counter
23 reg [3:0]cnt;
24 // block 4 PISO counter
25 reg [7:0] tx_shift_reg;
26 reg [3:0]cnt_r;
27
28
29
30 //-----current state FIRST BLOCK-----
31 always @(posedge clk or negedge rst_n) begin
32     if(!rst_n)
33         cs<=IDLE;
34     else
35         cs <= ns;
36 end

```

```

35 // -----next state SECOND BLOCK-----
36 always @(*) begin
37     ns = cs;
38     case (cs)
39
40         // IDLE
41         IDLE:
42             if(ss_n)
43                 ns = IDLE;
44             else
45                 ns = CHK_CMD;
46             // CHECK COMMAND
47             CHK_CMD: begin
48
49                 if (ss_n)
50                     ns = IDLE;
51
52                 else if (cmd_first == 1'b0 && MOSI == 1'b0)
53                     ns = WRITE; // 00
54                 else if (cmd_first == 1'b0 && MOSI == 1'b1)
55                     ns = WRITE; // 01
56                 else if (cmd_first == 1'b1 && MOSI == 1'b0)
57                     ns = READ_ADD; // 10
58                 else if (cmd_first == 1'b1 && MOSI == 1'b1)
59                     ns = READ_DATA; // 11
60
61             end
62
63             // WRITE
64             WRITE: begin
65                 if (!ss_n)
66                     ns = WRITE;
67                 else
68                     ns = IDLE;
69             end
70
71             // READ ADDRESS
72             READ_ADD:
73                 if(!ss_n)
74                     ns = READ_ADD;
75                 else
76                     ns = IDLE;

```

```

77         // READ DATA
78         READ_DATA:
79             if (!ss_n)
80                 ns = READ_DATA;
81             else
82                 ns = IDLE;
83             default:
84                 ns = IDLE;
85             endcase
86     end

```

```

92 // SIPO
93 always @(posedge clk) begin
94     if (!rst_n) begin
95         rx_data <= 10'd0;
96         cnt <= 4'd0;
97         rx_valid <= 1'b0;
98     end
99
100     else if (ss_n) begin
101         // Transaction finished
102         rx_data <= 10'd0;
103         cnt <= 4'd0;
104         rx_valid <= 1'b0;
105     end
106
107     else begin
108         // Receiving
109         rx_valid <= 1'b0;
110
111         if (cnt < 4'd10) begin
112             rx_data <= {rx_data[8:0], MOSI}; // shift left
113             if (cnt == 4'd9) begin
114
115                 rx_valid <= 1'b1;
116                 cnt <= 4'd10;
117             end
118             else begin
119                 cnt <= cnt + 1'b1; // COUNT FOR DATA INPUT
120             end
121         end
122     end
123 end

```

```

124 // command first (cmd)
125 always @(negedge clk) begin
126     if (!rst_n)
127         cmd_first <= 1'b0;
128
129     else if (ss_n)
130         cmd_first <= 1'b0;
131
132     else if (cs == CHK_CMD)
133         cmd_first <= MOSI; // load mosi
134 end
135
136 //----- PISO -----
137
138 always @(negedge clk ) begin
139     if (!rst_n) begin // reset
140         tx_shift_reg <= 8'd0;
141         cnt_r <= 4'd0;
142     end
143
144     else if (ss_n) begin // transaction finished
145         tx_shift_reg <= 8'd0;
146         cnt_r <= 4'd0;
147     end
148     else if (tx_valid && (cs == READ_DATA)) begin
149         tx_shift_reg <= tx_data; // load data
150         cnt_r <= 4'd0;
151     end
152
153     else if ((cs == READ_DATA) && (cnt_r < 4'd8)) begin
154         tx_shift_reg <= {tx_shift_reg[6:0], 1'b0}; // shift left
155         cnt_r <= cnt_r + 1'b1; // count for read data
156     end
157 end
158
159 assign MISO = (!ss_n && (cs == READ_DATA)) ? tx_shift_reg[7] : 1'b0; // miso output serial data
160 endmodule //SPI

```

3.5 SPI Slave Synthesis

The synthesis result represents how the SPI Slave RTL code is translated into a gate/register-level hardware implementation. Reviewing the RTL and synthesized schematics helps confirm that the intended logic (FSM, shift registers, counters) was correctly inferred by the synthesis tool.

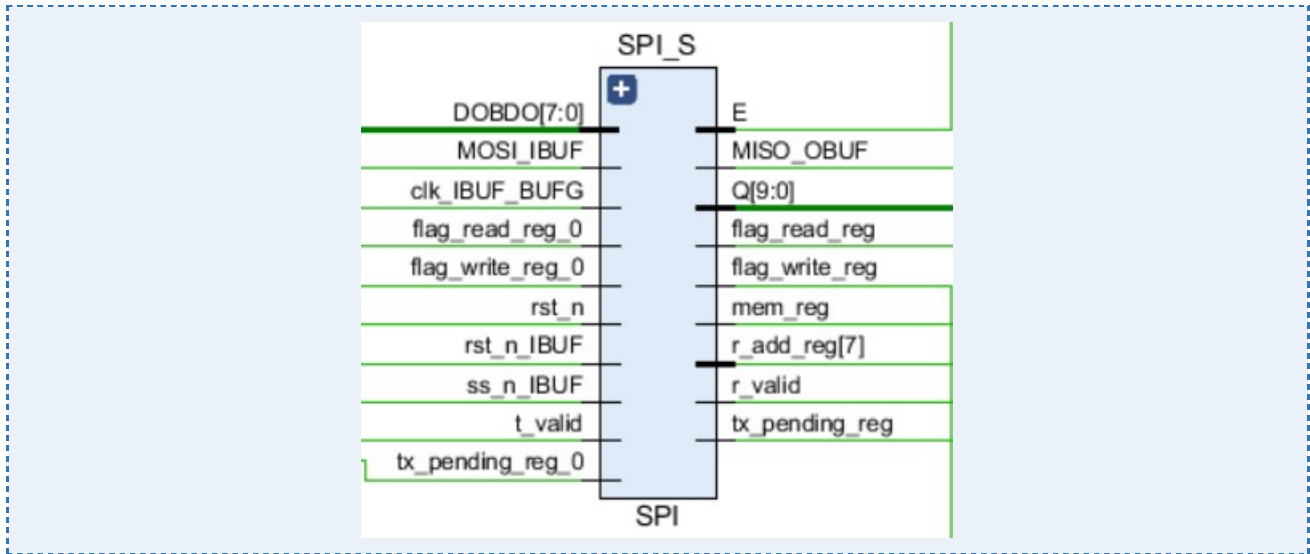


Figure 3.2 — RTL schematic of the SPI Slave module.

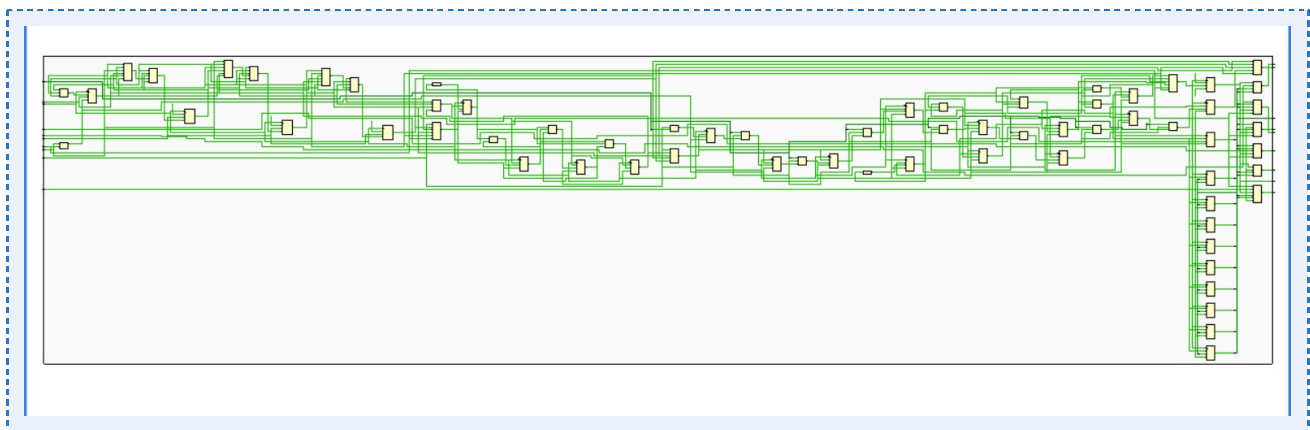


Figure 3.3 — Synthesized schematic of the SPI Slave module.

4. Single-Port RAM Module

The Single-Port RAM serves as the storage element of the system. It provides a single address/data interface that is shared between write and read operations, and is fully controlled by the SPI Slave module.

4.2 Relationship Between SPI Slave and RAM

- SPI Write Address → RAM Address
- SPI Write Data → RAM Data
- SPI Read Address → RAM Address
- RAM Data → SPI Slave → MISO

4.3 RAM RTL Code

```

1  module RAM #(parameter MEM_DEPTH = 256,parameter ADD_SIZE = 8)(
2      input  clk,rst_n,
3      input  [9:0]din,
4      input  rx_valid,
5      output reg [7:0]dout,
6      output reg tx_valid
7  );
8      reg [7:0] mem [MEM_DEPTH-1:0]; // memory declaration
9      reg [ADD_SIZE-1:0]w_add,r_add;    // addresses read - write
10
11     // pending operation
12     reg tx_pending; // tx operation
13     reg flag_read;
14     reg flag_write;
15
16     always @(posedge clk ) begin
17         if(!rst_n)begin // reset all signals
18             dout<=8'd0;
19             w_add<=8'd0;
20             r_add<=8'd0;
21             tx_valid<=1'b0;
22             flag_read<=0;
23             flag_write<=0;
24         end
25         else begin
26             tx_valid<=0;
27             if(tx_pending) begin
28                 tx_valid<=1;
29                 tx_pending<=0;
30             end
31             if(rx_valid) begin
32                 case ({din[9],din[8]})
33                     2'b00: begin // 00 write address
34                         if(!flag_write)begin
35                             w_add <= din[7:0];
36                             flag_write<=1;
37                         end
38                     end
39                     2'b01: begin // 01 write data
40                         if(flag_write)begin
41                             mem[w_add] <= din[7:0];
42                             flag_write<=0;
43                         end
44                     end
45                     2'b10: begin // read address
46                         if(!flag_read)begin
47                             r_add <= din[7:0];
48                             flag_read<=1;
49                         end
50                     end
51                     2'b11:begin // read data
52                         if(flag_read) begin
53                             dout <= mem[r_add];
54                             flag_read<=0;
55                             tx_pending<=1'b1;
56                         end
57                     end
58                 endcase
59             end
60         end
61     end
62 end
63 endmodule //RAM

```

Mem.dat

File	Edit	View
------	------	------

12
A5
3C
7F
CC
FF
81
55
AA
10
22
33
44
66
77
88
99
BC
DE
F0

4.4 RAM Synthesis

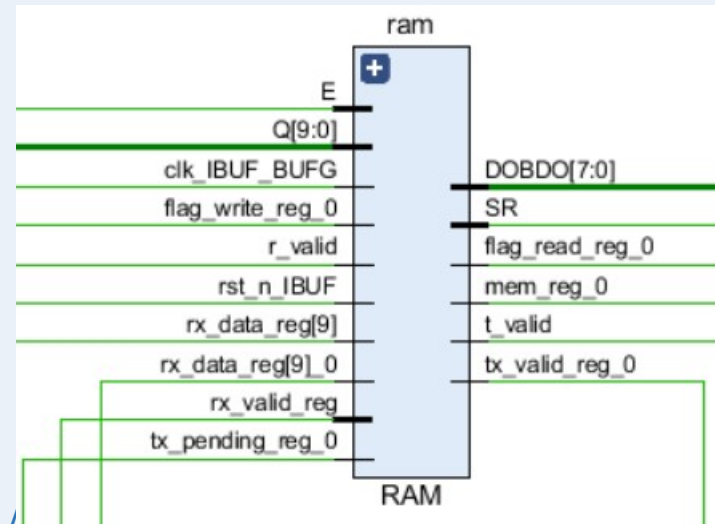


Figure 4.1 — RTL schematic of the Single-Port RAM module.

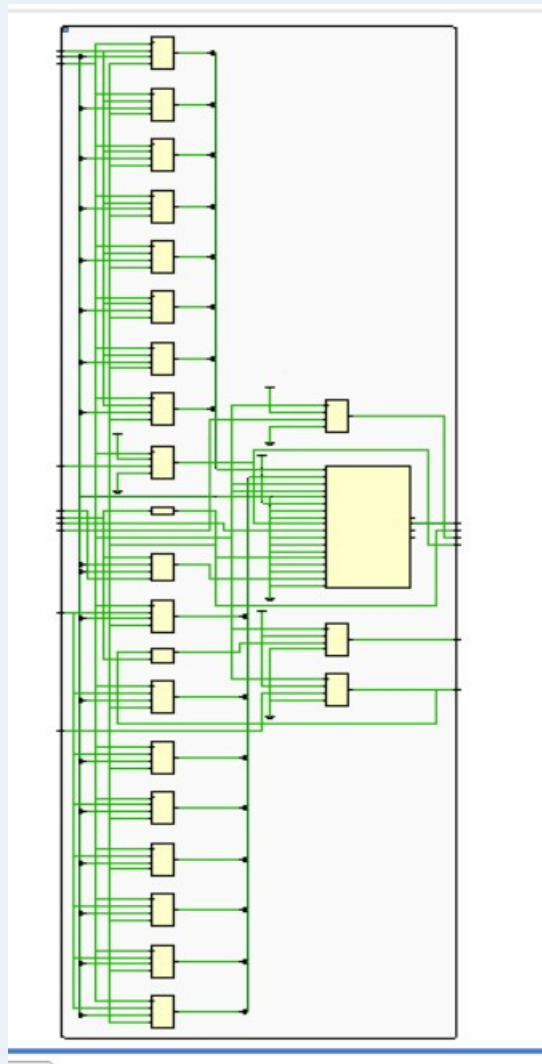


Figure 4.2 — Synthesized schematic of the Single-Port RAM module.

5. TOP Module

The TOP module connects the SPI Slave and the Single-Port RAM together, forming the complete top-level design. It routes the relevant address, data, and control signals between the two blocks and exposes the external SPI interface (MOSI, MISO, SS_N, CLK, RESET) to the outside world.

5.1 Module Connections

- SPI Slave → RAM Address
- SPI Slave → RAM Write Data
- RAM Read Data → SPI Slave
- SPI Slave → TX_VALID / RX_VALID (as applicable)

5.2 TOP RTL Code

```

2 > VERILOG > TOP.v > TOP
1  module TOP #(parameter MEM_DEPTH = 256 ,parameter ADD_SIZE = 8)(
2    input clk,rst_n,ss_n,
3    input MOSI,
4    output MISO
5  );
6  wire [9:0]rx;
7  wire r_valid;
8  wire [7:0]tx;
9  wire t_valid;
10 //-----*INSTANTIATION*-----
11 // SPI
12 SPI SPI_S (.clk(clk),
13            .rst_n(rst_n),
14            .ss_n(ss_n),
15            .MOSI(MOSI),
16            .MISO(MISO),
17            .rx_data(rx),
18            .rx_valid(r_valid),
19            .tx_data(tx),
20            .tx_valid(t_valid));
21
22 // RAM
23 RAM #(.MEM_DEPTH(MEM_DEPTH),
24       .ADD_SIZE(ADD_SIZE)
25       )
26 ram (.clk(clk),
27       .rst_n(rst_n),
28       .din(rx),
29       .rx_valid(r_valid),
30       .dout(tx),
31       .tx_valid(t_valid));
32 endmodule //TOP

```

5.3 TOP Synthesis

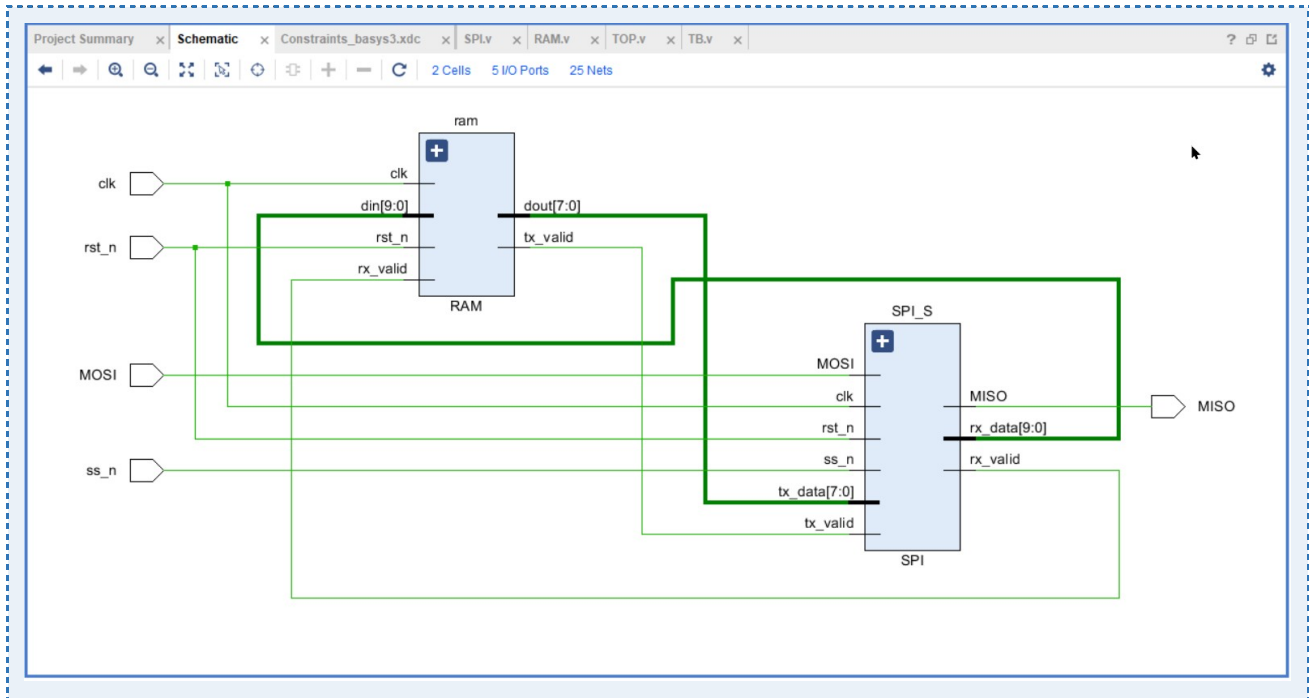


Figure 5.1 — RTL schematic of the TOP module.

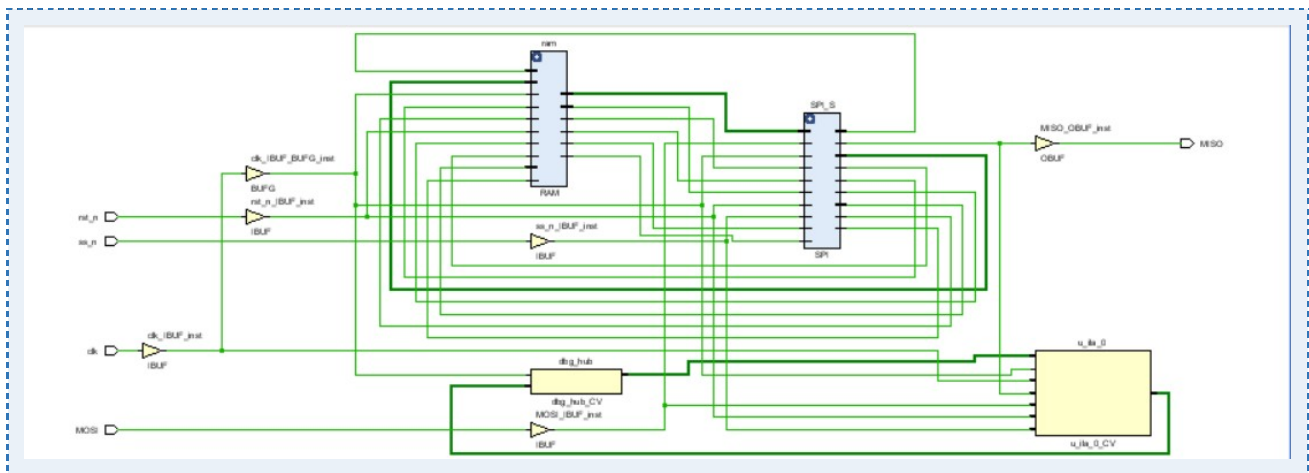


Figure 5.2 — Synthesized schematic of the TOP module.

6. Testbench

6.1 Purpose

The testbench is used to verify the functional correctness of the complete design (SPI Slave + RAM, via the TOP module) before proceeding to synthesis and implementation.

In the testbench, one write/read transaction is performed as follows

- Write 5 to address 15.
- Read address 15 → expected data 5.

6.3 Testbench RTL Code

```

1  module TB ();
2  // TOP
3  reg clk,rst_n,ss_n;
4  reg MOSI;
5  wire MISO;
6
7  //instantiate TOP
8  TOP DUT (.clk(clk),
9          .rst_n(rst_n),
10         .ss_n(ss_n),
11         .MOSI(MOSI),
12         .MISO(MISO)
13         );
14
15
16 //task for command check
17 task send_bit;
18     input command;
19     begin
20         MOSI = command;
21         @(negedge clk);
22     end
23 endtask
24 // task for address & data
25 task send_word;
26     input [7:0] word;
27     integer i;
28     begin
29         for (i = 7;i>=0 ;i = i - 1 ) begin
30             MOSI = word[i];
31             @(negedge clk);
32         end
33     end
34 endtask
35 //clk
36 initial begin
37     clk = 0;
38     forever
39     #1 clk = ~clk;
40 end

```

```

44 initial begin
45     // verify reset
46     rst_n=0;
47     $readmemh ("mem.dat",DUT.ram.mem);
48     ss_n=0;
49     MOSI=0;
50     repeat(4) @(negedge clk);
51     rst_n=1;
52 // WRITE ADDRESS
53 ss_n=0;
54 send_bit(0);
55 send_bit(0);
56 send_word(8'd15); // address
57 ss_n=1;
58 @(negedge clk);
59 @(negedge clk);
60 // WRITE DATA
61 ss_n=0;
62 send_bit(0);
63 send_bit(1);
64 send_word(8'd5); // data
65 ss_n=1;
66 @(negedge clk);
67 // READ ADDRESS
68 ss_n=0;
69 send_bit(1);
70 send_bit(0);
71 send_word(8'd15); // address
72 ss_n=1;
73 @(negedge clk);
74 // READ DATA
75 ss_n=0;
76 send_bit(1);
77 send_bit(1);
78 send_word(8'd15); // data
79 repeat (10) @(negedge clk);
80 ss_n=1;
81 #20;
82 $stop;
83 $display("communication is finshed");
84 end
85 endmodule //TB

```

```

85 initial begin
86     $monitor(
87         "TIME=%0t | ss_n=%b | rst_n=%b | MOSI=%b | MISO=%b ",
88         $time,
89         ss_n,
90         rst_n,
91         MOSI,
92         MISO
93     );
94 end
95 endmodule //TB

```

7. Simulation and Verification Results

7.1 Simulation 1 – Write Address and Write Data

This simulation verifies the write operation, in which the master first sends a Write Address command followed by a Write Data command:

- 00 → Write Address
- 01 → Write Data

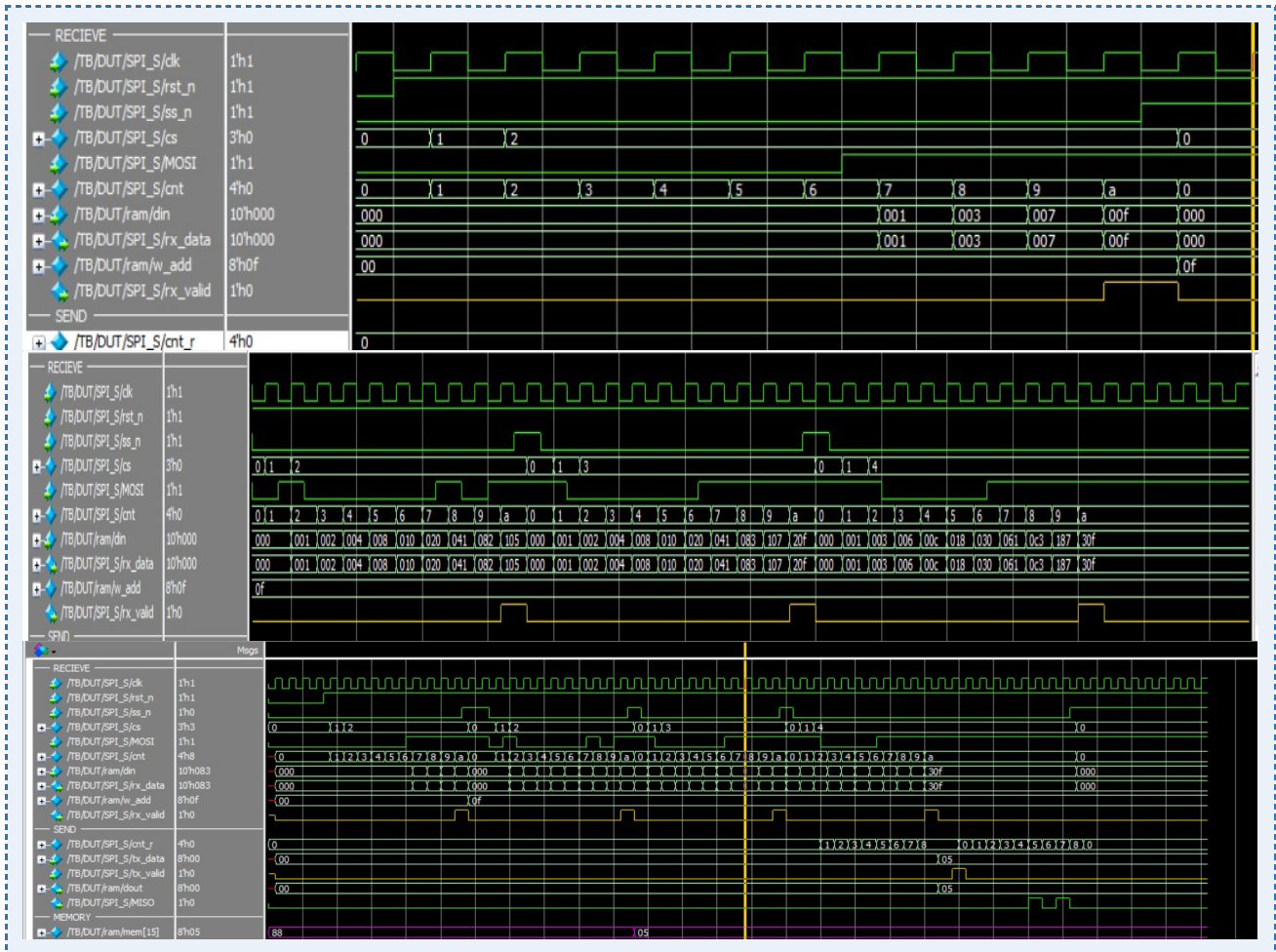


Figure 7.1 — Simulation waveform for Write Address and Write Data.

7.2 Simulation 2 – Read Address and Read Data

This simulation verifies the read operation, in which the master first sends a Read Address command followed by a Read Data command:

- 10 → Read Address
- 11 → Read Data

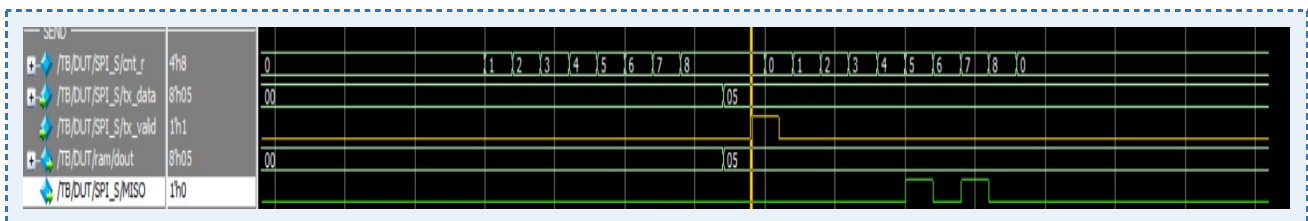


Figure 7.2 — Simulation waveform for Read Address and Read Data.

7.3 Simulation 3 – Multiple Write Operations

This simulation verifies that multiple addresses and data values can be written into RAM correctly in sequence, confirming that the SPI Slave and RAM handle repeated write transactions without errors.

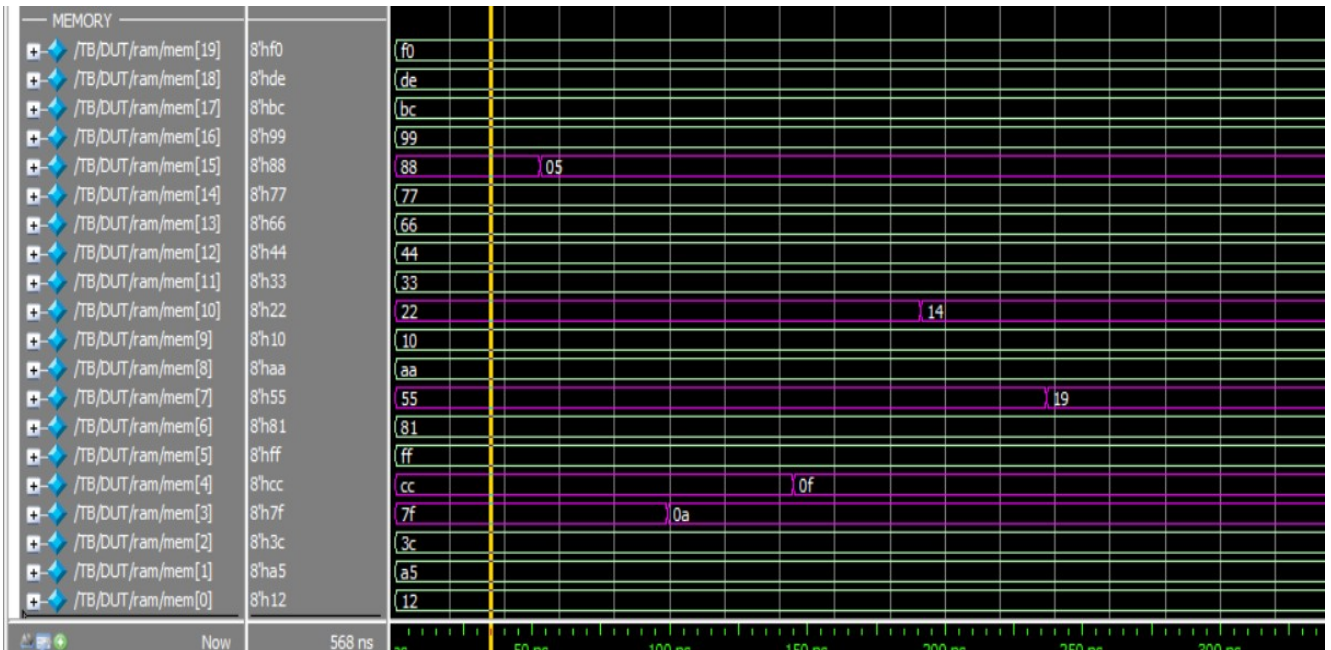


Figure 7.3 — Simulation waveform for Multiple Write Operations.

7.4 Simulation 4 – Multiple Write/Read Transactions

This simulation verifies that multiple complete write and read transactions can be performed sequentially, confirming correct end-to-end operation of the design:

- Write Address → Write Data → Read Address → Read Data

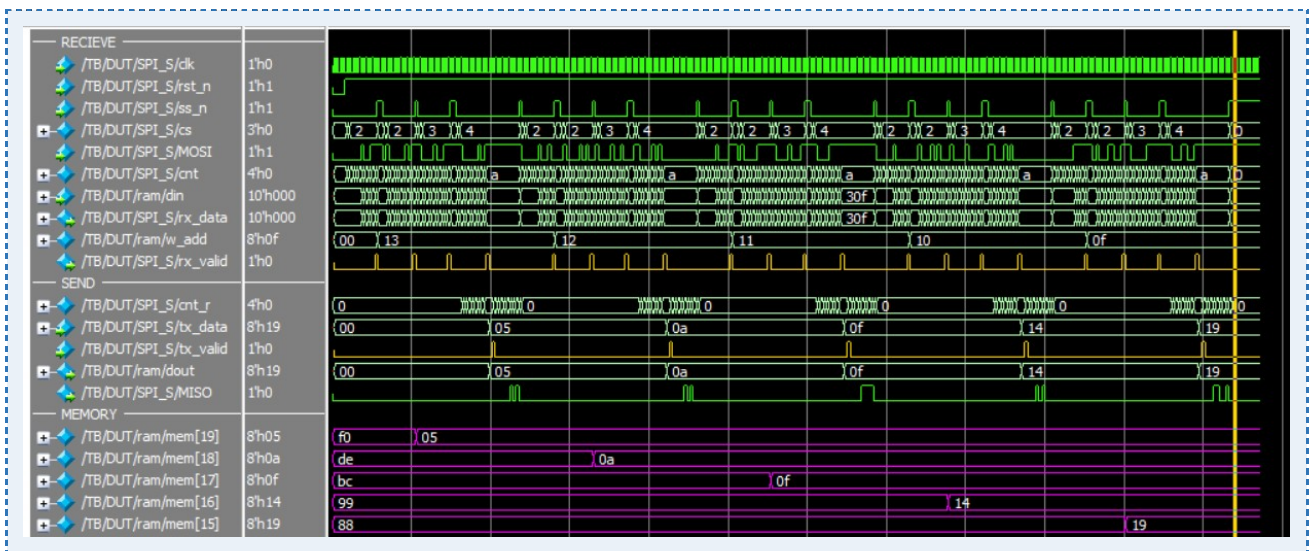


Figure 7.4 — Simulation waveform for Multiple Write/Read Transactions.

8. Elaboration

Elaboration is the stage in which the RTL description is checked and expanded into a hierarchical representation of the design, resolving module instantiations, parameters, and connectivity before synthesis begins..

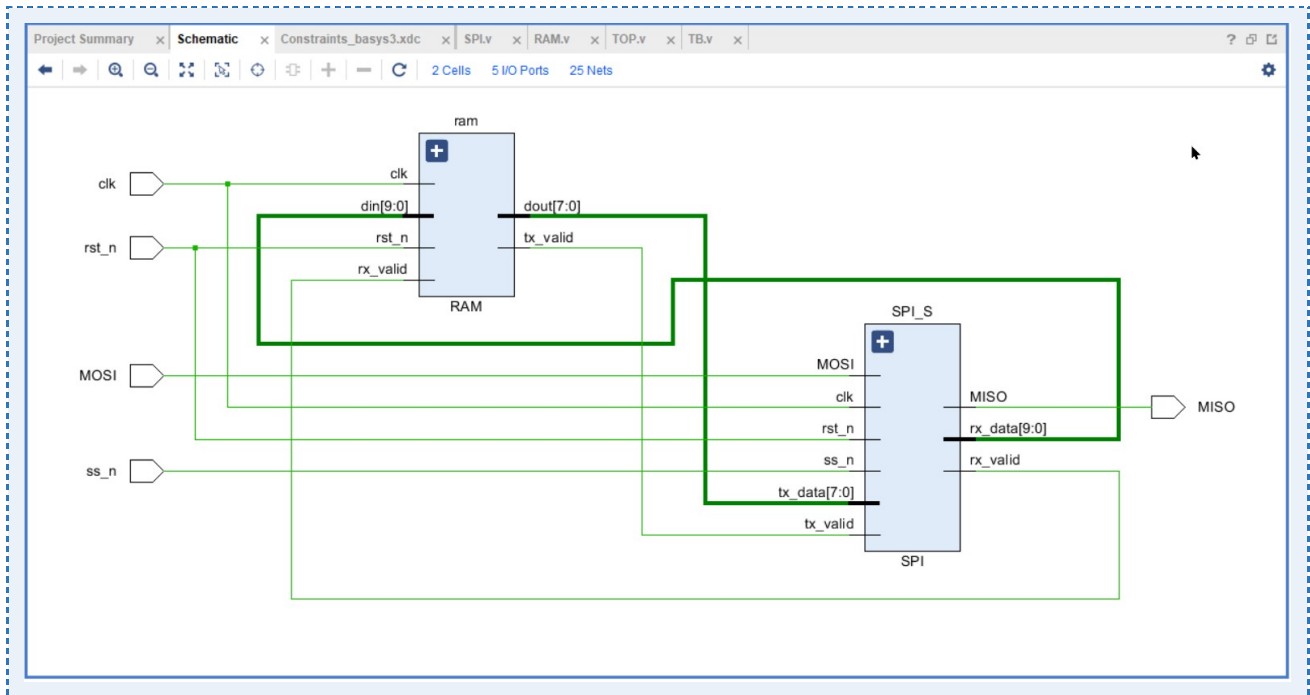


Figure 8.1 — Elaboration result / schematic.

9. Synthesis

Synthesis is the process of converting the RTL (Verilog) description of the design into a gate-level hardware representation composed of logic gates, flip-flops, and interconnects that can be mapped onto the target device.

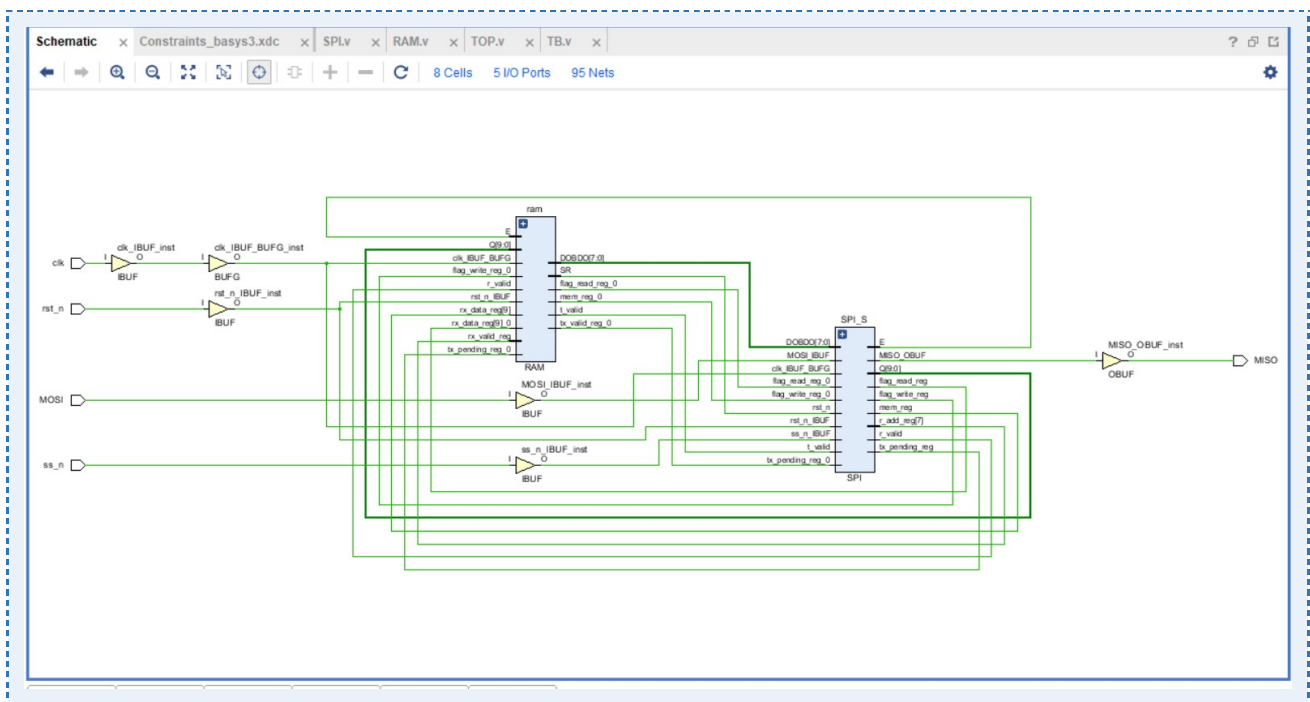
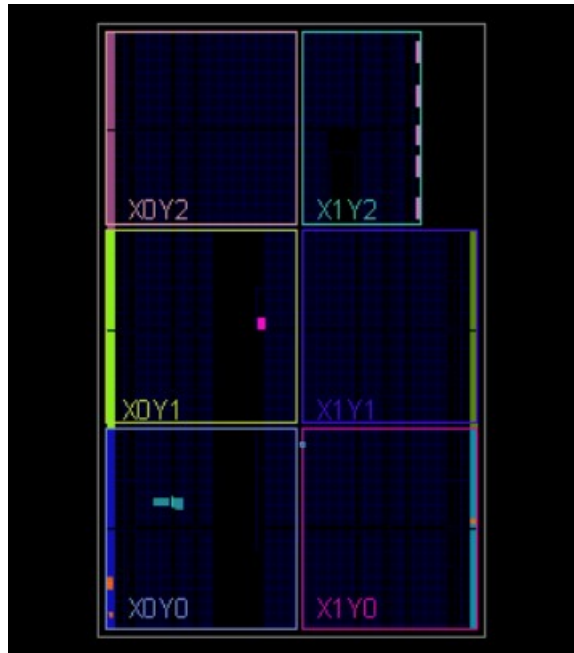


Figure 9.1 — Synthesized schematic of the complete design.

10. Implementation



11. Lintng tool

Severity	Status	Check	Alias	Message	Module	Category	State	Owner	STARC Reference
0	2	flop_with_inverted_clock		Inverted clock drives Flip-flop. Flop SPI_S.cmd_first, ...	SPI	Clock	open	unassign...	1.4.3.6
0	2	flop_with_inverted_clock		Inverted clock drives Flip-flop. Flop SPI_S.tx_shift_re...	SPI	Clock	open	unassign...	1.4.3.6
0	2	flop_with_inverted_clock		Inverted clock drives Flip-flop. Flop SPI_S.cnt_r, Mod...	SPI	Clock	open	unassign...	1.4.3.6
0	2	parameter_name_duplicate		Same parameter name is used in more than one mod...	RAM	Nomenclature...	open	unassign...	1.1.4.3, 3.2.2.2
0	2	parameter_name_duplicate		Same parameter name is used in more than one mod...	RAM	Nomenclature...	open	unassign...	1.1.4.3, 3.2.2.2
0	2	always_signal_assign_large		Always block has more signal assignments than the s...	RAM	Rtl Design Style	open	unassign...	2.6.1.3
0	2	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassign...	3.5.6.4, 3.5.6.6
0	2	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassign...	3.5.6.4, 3.5.6.6
0	2	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassign...	3.5.6.4, 3.5.6.6
0	2	comment_not_in_english		Comments include non-English characters. File C:/Us...	none	Nomenclature...	open	unassign...	3.5.6.4, 3.5.6.6
0	2	multi_ports_in_single_line		Multiple ports are declared in one line. Module SPI, FI...	SPI	Rtl Design Style	open	unassign...	3.5.6.3
0	2	multi_ports_in_single_line		Multiple ports are declared in one line. Module RAM, ...	RAM	Rtl Design Style	open	unassign...	3.5.6.3
0	2	multi_ports_in_single_line		Multiple ports are declared in one line. Module TOP, F...	TOP	Rtl Design Style	open	unassign...	3.5.6.3

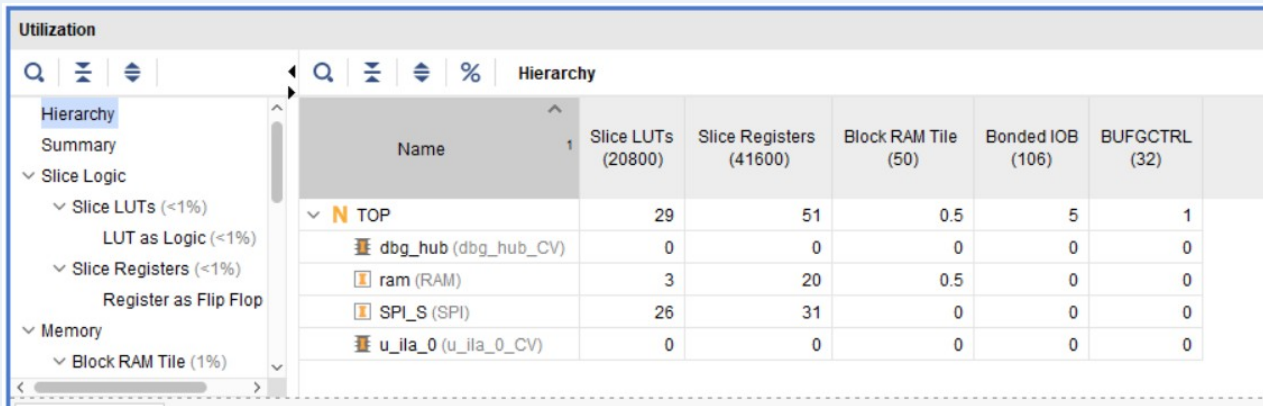
NO ERROR

BINARY ENCODING

12. Utilization Report

The utilization report summarizes the hardware resources consumed by the design on the target FPGA device, including logic elements, registers, block RAM, I/O, and clocking resources.

Table 10.1 — Resource Utilization

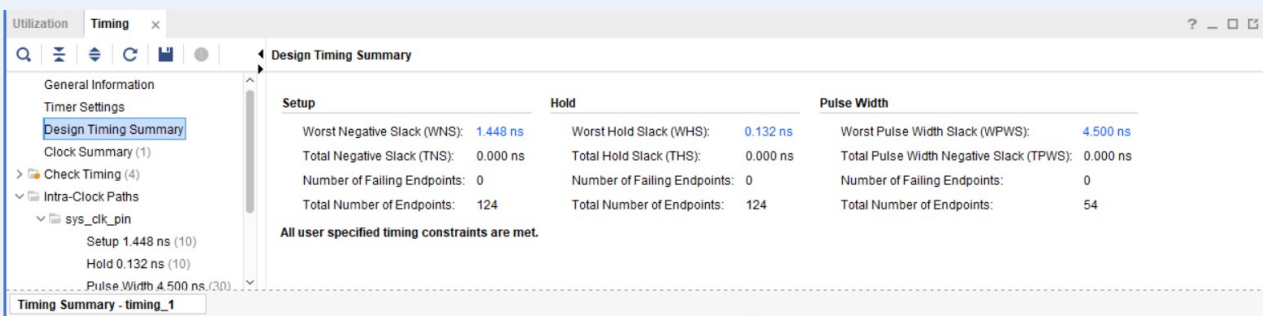


Name	Slice LUTs (20800)	Slice Registers (41600)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
TOP	29	51	0.5	5	1
dbg_hub (dbg_hub_CV)	0	0	0	0	0
ram (RAM)	3	20	0.5	0	0
SPI_S (SPI)	26	31	0	0	0
u_ila_0 (u_ila_0_CV)	0	0	0	0	0

Figure 12.1 — Utilization report screenshot.

13. Timing Report

Timing analysis checks whether the design meets the required timing constraints for correct operation at the target clock frequency.



Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.448 ns	Worst Hold Slack (WHS): 0.132 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 124	Total Number of Endpoints: 124	Total Number of Endpoints: 54

All user specified timing constraints are met.

Figure 13.1 — Timing report screenshot.

14. Power Report

Power analysis estimates the amount of power the design is expected to consume on the target device, broken down into dynamic and static components.

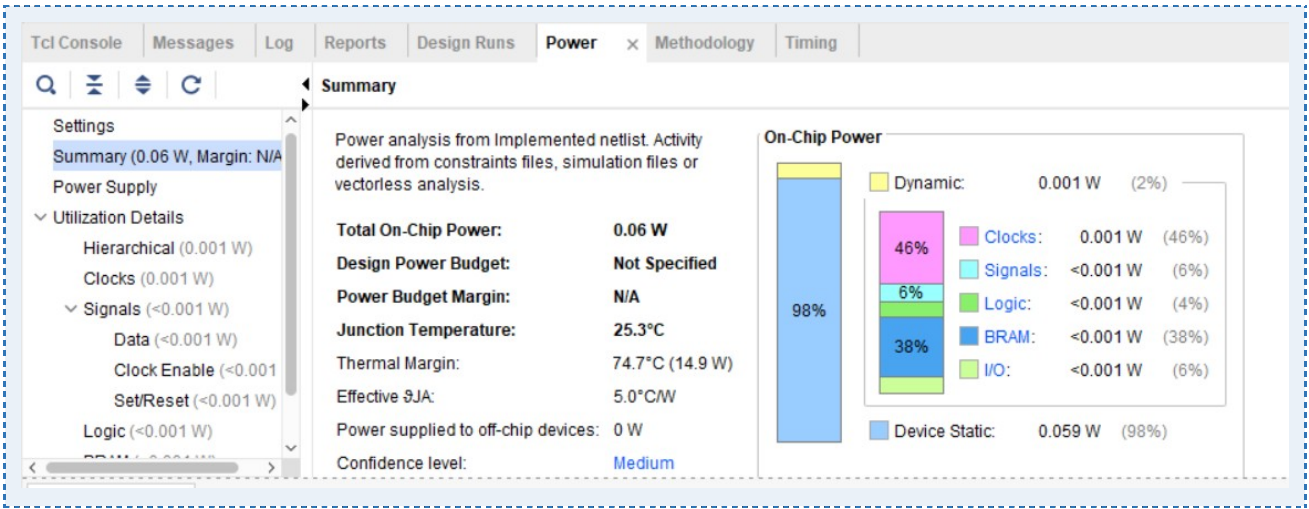


Figure 14.1 — Power report screenshot.

● GRAY ENCODING

	State	New Encoding	Previous Encoding
105			
106			
107			
108	IDLE	000	000
109	CHK_CMD	001	001
110	WRITE	011	010
111	READ_ADD	010	011
112	READ_DATA	111	100
113			

Utilization Report:

Tcl Console	Messages	Log	Reports	Design Runs	Utilization	Debug
Hierarchy						
Hierarchy						
Summary						
v Slice Logic						
v Slice LUTs (<1%)						
LUT as Logic (<1%)						
v Slice Registers (<1%)						
Register as Flip Flop (v						
Name						
Slice LUTs (20800)						
Slice Registers (41600)						
Block RAM Tile (50)						
Bonded IOB (106)						
BUFGCTRL (32)						
v TOP						
dbg_hub (dbg_hub_CV)						
ram (RAM)						
SPL_S (SPI)						

Timing Report:

Tcl ConsoleMessagesLogReportsDesign RunsTimingUtilizationDebug

Q

Design Timing Summary

General Information

Timer Settings

Design Timing Summary

Clock Summary (1)

Check Timing (4)

Intra-Clock Paths

Inter-Clock Paths

Other Path Groups

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.448 ns	Worst Hold Slack (WHS): 0.132 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 124	Total Number of Endpoints: 124	Total Number of Endpoints: 54

All user specified timing constraints are met.

Timing Summary - timing_1

Figure — Timing report screenshot.

● ONE_HOT_ENCODING

State	New Encoding	Previous Encoding
IDLE	00001	000
CHK_CMD	00010	001
WRITE	00100	010
READ_ADD	01000	011
READ_DATA	10000	100

Utilization Report:

Tcl ConsoleMessagesLogReportsDesign RunsMethodologyTimingUtilization x

Q

≡

⚙

Q

≡

⚙

%

Hierarchy

Hierarchy

Summary

▼ Slice Logic

▼ Slice LUTs (<1%)

LUT as Logic (<1%)

▼ Slice Registers (<1%)

Register as Flip Flop (▼)

Name	Slice LUTs (20800)	Slice Registers (41600)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	Block RAM Tile (50)	Bonded IOB (106)	BUFGCTRL (32)
▼ N TOP	29	51	13	29	17	0.5	5	1
ram (RAM)	3	20	6	3	1	0.5	0	0
SPI_S (SPI)	26	31	11	26	12	0	0	0

utilization 4

Timing Report:

Tcl ConsoleMessagesLogReportsDesign RunsMethodologyTiming xUtilization? _ □ □

Q⏏⌂⌂⌂⌂⌂

Design Timing Summary

General Information
Timer Settings
Design Timing Summary
Clock Summary (1)
Check Timing (4)
Intra-Clock Paths
Inter-Clock Paths
Other Path Groups

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.460 ns	Worst Hold Slack (WHS): 0.102 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 124	Total Number of Endpoints: 124	Total Number of Endpoints: 54

All user specified timing constraints are met.

Timing Summary - timing_1

Figure — Timing report screenshot.

RESULTS

- **Binary:** Balanced area and timing.
- **Gray:** Reduced switching activity and good area efficiency.
- **One-Hot:** Faster timing with higher area usage.

13. Final Project Results / Conclusion

This project implemented and verified the following:

- SPI communication was implemented between an external SPI Master interface and the internal SPI Slave module.
- SPI commands (Write Address, Write Data, Read Address, Read Data) were decoded by the SPI Slave FSM.
- RAM write/read operations were implemented and controlled through the SPI Slave.
- The SPI Slave and RAM were integrated using the TOP module.
- The design was verified using simulation with a dedicated testbench covering multiple scenarios.
- Elaboration and synthesis were performed to translate the RTL into a hardware representation.
- Utilization, timing, and power were analyzed to characterize the implemented design.